Week 9 Lab: PostgreSQL & SQLModel

Part 1 — SQL Warm-up: Tables, Keys, Relationships

1.2

Task 1. Write SQL queries for:

1. All heroes aged 18 or older, oldest first.

```
appdb=> SELECT *
appdb-> FROM hero
appdb-> WHERE age >= 18
appdb-> ORDER BY age DESC;
 id |  name   | age | team_id
----+---------+-----+---------
  3 | Logan   | 150 |       2
  1 | Tony    |  45 |       1
  2 | Natasha |  35 |       1
(3 rows)
```

2. Each hero's name next to their team's name (heroes without a team must also appear).

```
appdb=> SELECT hero.name AS hero_name,
appdb->        team.name AS team_name
appdb-> FROM hero
appdb-> LEFT JOIN team ON hero.team_id = team.id;
 hero_name | team_name
-----------+-----------
 Tony      | Avengers
 Natasha   | Avengers
 Logan     | X-Men
 Peter     |
(4 rows)
```

3. The number of heroes per team.

```
appdb=> SELECT team.name,
appdb->        COUNT(hero.id) AS hero_count
appdb-> FROM team
appdb-> LEFT JOIN hero ON team.id = hero.team_id
appdb-> GROUP BY team.id, team.name;
   name   | hero_count
----------+------------
 X-Men    |          1
 Avengers |          2
(2 rows)
```

4. The 2nd page of heroes when a page has 2 rows, ordered by id .

```
appdb=> SELECT *
appdb-> FROM hero
appdb-> ORDER BY id
appdb-> LIMIT 2 OFFSET 2;
 id | name  | age | team_id
----+-------+-----+---------
  3 | Logan | 150 |       2
  4 | Peter |  16 |
(2 rows)
```

1.3

```
appdb=> INSERT INTO team (name, headquarters) VALUES ('Avengers', 'Los Angeles');
ERROR:  duplicate key value violates unique constraint "team_name_key"
DETAIL:  Key (name)=(Avengers) already exists.
appdb=> INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);
ERROR:  insert or update on table "hero" violates foreign key constraint "hero_team_id_fkey"
DETAIL:  Key (team_id)=(99) is not present in table "team".
appdb=> INSERT INTO hero (age) VALUES (30);
ERROR:  null value in column "name" of relation "hero" violates not-null constraint
DETAIL:  Failing row contains (6, null, 30, null).
appdb=> DELETE FROM team WHERE id = 1;
ERROR:  update or delete on table "team" violates foreign key constraint "hero_team_id_fkey" on table "hero"
DETAIL:  Key (id)=(1) is still referenced from table "hero".
```

Question 1: For each of the four statements, which constraint blocked it (PRIMARY KEY , UNIQUE , NOT NULL , FOREIGN KEY) and why?

1. `INSERT INTO team ... ('Avengers', ...)`

- Constraint: `UNIQUE`
- Reason: `team.name` is unique, and `Avengers` already exists.

2. `INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);`

- Constraint: `FOREIGN KEY`
- Reason: `team_id = 99` does not exist in the `team` table.

3. `INSERT INTO hero (age) VALUES (30);`

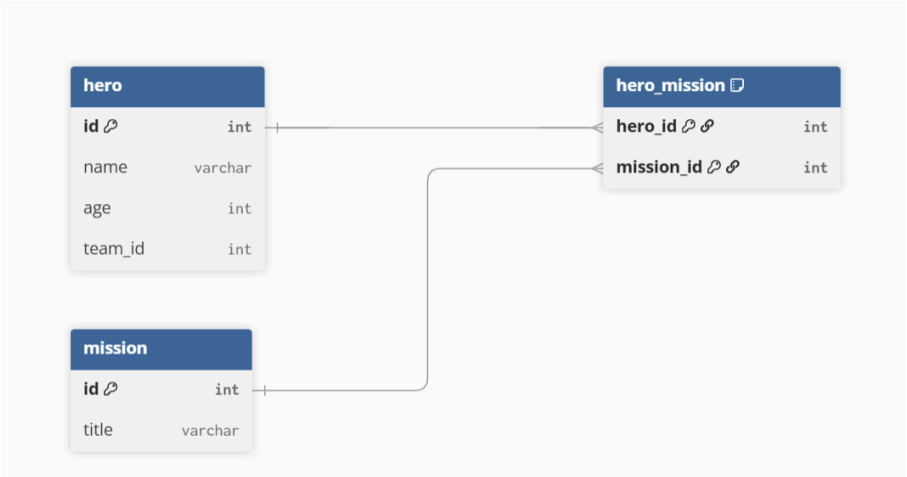
- Constraint: `NOT NULL`
- Reason: The required `name` field was not provided.

4. `DELETE FROM team WHERE id = 1;`

- Constraint: `FOREIGN KEY`
- Reason: Some heroes still reference `team.id = 1`, so the team cannot be deleted.

Question 2. The relationship team → hero is one-to-many. Why is the foreign key on hero and not on team ?

Because one team can have many heroes, but each hero belongs to at most one team. Therefore, each hero stores a `team_id` foreign key that references `team(id)`
Question 3. Heroes can go on many missions and a mission has many heroes (many-to-many). Sketch the tables you need (names, columns, PK, FK). Hint: you need a link table



1.4 Clean up

```
appdb=> DROP TABLE hero;
DROP TABLE
appdb=> DROP TABLE team;
DROP TABLE
appdb=> \q
```

Part 2 — Project Setup and DATABASE_URL

checkpoint 2:

```
(.venv) PS C:\Users\XPS\hero-api> $env:DATABASE_URL = "postgresql+psycopg://app:secret@localhost:5432/appdb"
(.venv) PS C:\Users\XPS\hero-api> echo $env:DATABASE_URL
postgresql+psycopg://app:secret@localhost:5432/appdb
(.venv) PS C:\Users\XPS\hero-api> python -c "import sqlmodel, psycopg"
(.venv) PS C:\Users\XPS\hero-api>
```

Question 4. Why read the URL from an environment variable instead of writing it in database.py ? Give two reasons

We read DATABASE_URL from an environment variable because:
1. It keeps database credentials out of the source code.
2. It allows different database configurations without changing the code.

Part 3 — Define the Models

3.1

Question 5. Why is id typed int | None with default=None , when every row in the database has an id?
- Because before a new object is inserted into the database, it does not have an `id` yet. Therefore `id` can be `None` at first. After the row is inserted, the database automatically generates the `id`.

Question 6. Which attributes of Hero become columns, and which one does not? What is back_populates for?

- The attributes `id`, `name`, `age`, `secret_name`, and `team_id` become columns in the `hero` table.

- The `team` attribute does **not** become a column because it is a `Relationship`.

- `back_populates` is used to connect both sides of the relationship, so `Hero.team` links to `Team.heroes` and vice versa

3.2

```
(.venv) PS C:\Users\XPS\hero-api> python -c "from app.models import Hero, HeroPublic; print(Hero.__tablename__)"
hero
```

Part 4 - Engine, Tables and Session

4.1

```
(.venv) PS C:\Users\XPS\hero-api> python -c "from app.database import engine, SessionDep; print(engine.url)"
postgresql+psycopg://app:***@localhost:5432/appdb
```

4.2

```
PS C:\Users\XPS> psql "postgresql://app:secret@localhost:5432/appdb" -c "\dt"
        List of tables
 Schema | Name | Type  | Owner
--------+------+-------+-------
 public | hero | table | app
 public | team | table | app
(2 rows)


PS C:\Users\XPS> psql "postgresql://app:secret@localhost:5432/appdb" -c "\d hero"
                                  Table "public.hero"
   Column    |       Type        | Collation | Nullable |              Default
-------------+-------------------+-----------+----------+-----------------------------------
 name        | character varying |           | not null |
 age         | integer           |           |          |
 team_id     | integer           |           |          |
 id          | integer           |           | not null | nextval('hero_id_seq'::regclass)
 secret_name | character varying |           | not null |
Indexes:
    "hero_pkey" PRIMARY KEY, btree (id)
    "ix_hero_name" btree (name)
Foreign-key constraints:
    "hero_team_id_fkey" FOREIGN KEY (team_id) REFERENCES team(id)
```

Question 7. Compare the CREATE TABLE hero printed by SQLAlchemy with the one you wrote by hand in Part 1. List the differences (types, NOT NULL , indexes, constraints).

Compared with the table created manually in Part 1:

- The SQLModel version adds the `secret_name` column.
- `id` is an `INTEGER` primary key with an auto-generated sequence.
- `name` is `VARCHAR NOT NULL` and has an index.
- `age` is `INTEGER` and nullable.
- `team_id` is `INTEGER` and nullable.
- `team_id` has a `FOREIGN KEY` referencing `team(id)`.
- `secret_name` is `VARCHAR NOT NULL`

Question 8. Stop and restart the server. Is CREATE TABLE printed again? Why? What does create_all do when a table already exists?

No, `CREATE TABLE` is not executed again for tables that already exist. `create_all()` checks whether the tables already exist and only creates tables that are missing.

Question 9. create_all only knows about models that have been imported. Which line in main.py makes sure Hero and Team are registered?

- The line: from app.models import Hero, Team: makes sure `Hero` and `Team` are imported and registered in `SQLModel.metadata`

Part 5 - CRUD Endpoints for Heroes

```
PS C:\Users\XPS> curl.exe -X POST "http://localhost:8000/heroes" -H "Content-Type: application/json" -d '{\"name\":\"Pet
er\",\"age\":16,\"secret_name\":\"Spider-Man\"}'
{"name":"Peter","age":16,"team_id":null,"id":1}
```

5.1

Question 10. Comment out session.commit() in create_hero and create a hero. What does the response look like, and is the row in the database (SELECT * FROM hero;)? Put the line back. What does add() do on its own, and why do we need refresh() ?

- If `session.commit()` is commented out, the response may still return the new hero object, but the row is **not permanently saved** in the database.
- `session.add()` only adds the object to the current session; it does not save it permanently.
- `session.commit()` writes the transaction to the database.
- `session.refresh()` reloads the object from the database so generated values, such as `id`, are updated in the Python object.

Question 11. Which SQL statement does echo=True print for PATCH with body {"age": 17} ? Does it update every column or only age ? Why?

- For: {"age": 17}, `echo=True`: prints an SQL statement similar to: UPDATE hero SET age = %(age)s WHERE hero.id = %(hero_id)s;
- It updates **only the age column**, not every column.
- This happens because: hero_in.model_dump(exclude_unset=True), returns only the fields actually sent by the client, and `sqlmodel_update()` updates only those fields.

Question 12. Look at the JSON returned by GET /heroes/{id} . Is secret_name there? Which line of code is responsible?

- No, `secret_name` is **not** included in the JSON returned by: GET /heroes/{id}
- The responsible line is: response_model=HeroPublic
- For example:@app.get("/heroes/{hero_id}", response_model=HeroPublic)
- `HeroPublic` does not contain `secret_name`, so FastAPI excludes it from the response.

5.2

```
PS C:\Users\XPS> Invoke-RestMethod -Method GET -Uri "http://localhost:8000/heroes"

name    age team_id id
----    --- ------- --
Peter    16       2
Tony     45       3
Natasha  35       4
Logan   150       5
```

Part 6 — Teams, Filtering, Pagination and Errors

6.1

First POST /teams with name "Avengers":
→ Team created successfully.

Second POST /teams with the same name "Avengers":
→ 409 Conflict
→ {"detail":"Team name already exists"}

```
PS C:\Users\XPS> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/teams" -ContentType "application/json" -Body '{"name":"Av
engers","headquarters":"New York"}'

name     headquarters id
----     ------------ --
Avengers New York      1


PS C:\Users\XPS> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/teams" -ContentType "application/json" -Body '{"name":"Av
engers","headquarters":"New York"}'
Invoke-RestMethod : {"detail":"Team name already exists"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:8000/teams" -Co ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExc
   eption
```

6.2

Question 13. Call GET /heroes?min_age=18&team_id=1 and copy the SELECT printed by echo=True . Where do the values 18 and 1 appear? Why is this safe against SQL injection?

- The values 18 and 1 appear as bound parameters in the SQL query, not directly inside the SQL string.
- This is safe against SQL injection because SQLAlchemy uses parameterized queries and sends the values separately from the SQL statement.

Question 14. Why filter in the database instead of [h for h in session.exec(select(Hero)).all() if h.age >= 18] ?

Filtering in the database is better because the database only returns the rows we need. It uses less memory, transfers less data, and lets PostgreSQL optimize the query efficiently6.3

```
PS C:\Users\XPS> Invoke-RestMethod -Method GET -Uri "http://localhost:8000/heroes?min_age=18&team_id=1"
PS C:\Users\XPS> Invoke-RestMethod -Method POST -Uri "http://localhost:8000/heroes" -ContentType "application/json" -Body '{"name":"G
host","age":30,"secret_name":"Ghost","team_id":999}'
Invoke-RestMethod : {"detail":"Team not found"}
At line:1 char:1
+ Invoke-RestMethod -Method POST -Uri "http://localhost:8000/heroes" -C ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\XPS> Invoke-RestMethod -Method PATCH -Uri "http://localhost:8000/heroes/2" -ContentType "application/json" -Body '{"team_
id":999}'
Invoke-RestMethod : {"detail":"Team not found"}
At line:1 char:1
+ Invoke-RestMethod -Method PATCH -Uri "http://localhost:8000/heroes/2" ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\XPS> Invoke-RestMethod -Method GET -Uri "http://localhost:8000/heroes"

name    age team_id id
----    --- ------- --
Peter    16       2
Tony     45       3
```

Part 7 — Many-to-Many: Missions

```
PS C:\Users\XPS> psql "postgresql://app:secret@localhost:5432/appdb" -c "SELECT * FROM heromissionlink;"
 hero_id | mission_id
---------+------------
       2 |          1
(1 row)
```

Question 15. On restart, create_all did create mission and heromissionlink . In Part 4 it did nothing for hero . What is the rule?

The rule is that SQLModel.metadata.create_all(engine) creates only missing tables. It does not recreate or modify tables that already exist. Therefore, mission and heromissionlink were created on restart, while hero was left unchanged.

Part 8 — Seed Script

```
PS C:\Users\XPS> Invoke-RestMethod -Method GET -Uri "http://localhost:8000/teams"

name     headquarters id
----     ------------ --
Avengers New York      1
X-Men    Westchester   2
```

Question 16. You never set team_id in the seed script. Read the echo=True output: in which order were the INSERT s executed, and how did hero.team_id get its value?

The `INSERT` statements were executed in the order: `team` → `mission` → `hero` → `heromissionlink`.

`hero.team_id` was filled automatically through the relationship, for example `team=avengers`. SQLModel/SQLAlchemy first inserted the team, got its generated `id`, and then used that value as the hero's `team_id`. Therefore, `team_id` did not need to be set manually.

Part 9 — Migrations with Alembic

Question 17. Is there a power column? Now call GET /heroes . What happens and why? Why is "drop all tables and run create_all again" not an acceptable fix in production?

No, the power column is not created just by restarting the application.
- create_all() only creates missing tables; it does not modify an existing table to add a new column.
- GET /heroes fails because the model expects the power column but the database schema does not have it.
Dropping all tables is not acceptable in production because it would delete existing data.

Question 18. Copy the bodies of upgrade() and downgrade() . What does each one do?
- The `upgrade()` function adds the new `power` column to the `hero` table.
- The `downgrade()` function removes the `power` column from the `hero` table and restores the database schema to the previous version.

Question 19. Where does Alembic store "which revision this database is at"? (Hint: \dt .) Why should the migrations/ folder be committed to Git?

- Alembic stores the current database revision in the `alembic_version` table.

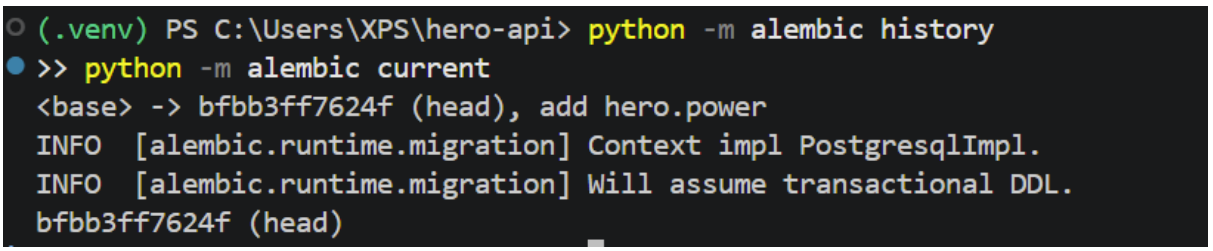
- The `migrations/` folder should be committed to Git because it contains the history of database schema changes. This allows every developer and deployment environment to apply the same migrations in the same order and keep the database schema consistent.

Question 20. Rename secret_name to alias in the model and run revision --autogenerate (do not apply it). What did Alembic generate? Why is that dangerous for existing data, and how would you fix the script? Afterwards, delete that revision file and undo the rename.

- Alembic generated a migration that added a new `alias` column and removed the old `secret_name` column instead of recognizing it as a column rename.

- This is dangerous because dropping `secret_name` can cause the existing data in that column to be lost.

- The migration should be edited manually so that the existing column is renamed from `secret_name` to `alias` instead of dropping and recreating it. After reviewing it, delete that test revision file and change `alias` back to `secret_name` in the model.



Review Questions

1. What is a primary key for?
- A primary key uniquely identifies each row in a table
2. What is the main benefit of an ORM?
- An ORM lets you work with database tables using Python objects and methods instead of writing SQL for every operation
3. What does table=True change about a SQLModel class?
- It makes the SQLModel class represent a real database table. Without `table=True`, it is only used as a data/schema model
4. After session.add(...) , what actually saves the data to the database?
- `session.commit()` saves the changes to the database
5. What does session.exec(select(Model)).all() return?
- It returns a list of all matching model objects from the database
6. What is Alembic for?
- Alembic is used to manage and apply database schema migrations, such as adding, removing, or changing columns and tables.
7. You add a field to a model and restart the app. Why is there no new column?
- Because `create_all()` only creates missing tables; it does not modify existing tables to add new columns. A migration is needed.
8. In a one-to-many Team–Hero relationship, where does the foreign key go?
- The foreign key goes on the “many” side, so `hero.team_id` references `team.id`.